SENTIMENT ANALYSIS SERVICE

Project Mastery Handbook

Implementation-first, code-backed, onboarding-oriented

Ownership Model

Role	Primary Responsibility
Tech Lead	Architecture governance, technical roadmap, security review, production readiness
ML Engineering Lead	Training pipeline, dataset strategy, evaluation, LoRA/PEFT, MLflow, DVC
AI Engineering Lead	Inference runtime, ONNX, SHAP, ABSA, language detection, prediction pipeline
Solution Engineering Lead	End-to-end solution design, business flow, integration, monitoring, deployment
Backend Engineering Lead	FastAPI, API contracts, validation, batch processing, evaluation APIs
UI/UX Engineering Lead	Angular frontend, explainability UI, batch upload UI, visualization

Source of Truth

Source code first → runtime configuration → tests → existing reports → documentation last

Contents

1 Executive Summary **2**

2 Business Context **2**
2.1 Who uses it . . . 2
2.2 What success looks like . . . 3
2.3 Important mismatch . . . 3

3 System Mental Model **3**
3.1 One-page view . . . 3
3.2 Engineer view . . . 4

4 Architecture Deep Dive **4**
4.1 Runtime architecture . . . 4
4.2 Design tradeoffs . . . 5

5 Runtime Execution Flow **5**
5.1 Application startup . . . 5
5.2 Prediction flow . . . 5
5.3 Other flows . . . 5

6 Frontend Deep Dive **6**
6.1 Component hierarchy . . . 6
6.2 State and storage . . . 6
6.3 Frontend/backend contract risks . . . 6

7 Backend Deep Dive **6**
7.1 API surface . . . 7
7.2 Error handling . . . 7

8 AI Inference Deep Dive **7**
8.1 What the model layer does . . . 7
8.2 Core concepts . . . 7
8.3 Production implications . . . 8

9 ML Training Deep Dive **8**
9.1 Training pipeline . . . 8
9.2 Balancing and loss . . . 8

10 Data Lineage **9**

11 DVC Pipeline **9**

12 ONNX Pipeline **10**

13 API Reference **10**

14 Security Review **10**

15 Observability **11**

16 Performance Analysis **11**

17 Failure Analysis **11**

18 Technical Debt Register **11**

19 Ownership Matrix **12**

20 RACI Matrix **12**

21 Team Responsibilities **12**

22 Debugging Handbook **12**

22.1 Backend does not start 12

22.2 Prediction fails 13

22.3 Batch upload fails 13

23 Incident Response Guide **13**

24 New Engineer Onboarding Guide **13**

25 Production Readiness Assessment **13**

26 Roadmap **14**

1 Executive Summary

This repository is a containerized sentiment analysis product plus the offline ML system that produces its model artifacts. The online path is Angular → Nginx → FastAPI → model inference. The offline path is DVC → preprocessing → finetuning → evaluation → ONNX export. The system also exposes monitoring, experiment tracking, and explainability.

The project exists because text feedback is valuable, but manual review is slow and expensive. The codebase turns raw text into structured outputs: overall sentiment, aspect-level sentiment, sarcasm flag, and token-level explanations. That makes it useful for product analytics, support triage, and demos of explainable AI.

Why an experienced engineer would build it this way:

- the application separates serving from artifact production, so runtime can stay lightweight
- a single backend process keeps deployment simple
- PEFT/LoRA keeps training and artifact sizes manageable
- ONNX gives a more predictable production inference path
- DVC and MLflow create reproducibility across data, training, and evaluation

2 Business Context

2.1 Who uses it

- end users who want a quick sentiment answer
- analysts who want to review a CSV in bulk
- ML engineers who train and export adapters
- backend engineers who maintain API behavior
- platform engineers who operate the containers and metrics

2.2 What success looks like

Table 1: Success signals

Area	Signal	Why it matters
Product	Accurate sentiment and ABSA output	User trust and business value
Runtime	Low latency on `/predict` and `/batch_predict`	Interactive use stays fast
Reliability	Health endpoint and graceful failure handling	Deployments can be monitored
ML	Reproducible training and evaluation runs	Models can be re-created safely
Operations	Prometheus, Grafana, MLflow, logs	Operators can debug and observe

2.3 Important mismatch

The frontend includes a language selector, but `AppComponent` currently drops the selected language before sending the request. So the UI suggests explicit language control, while the backend usually relies on detection. That is an implementation mismatch, not just a docs issue.

3 System Mental Model

3.1 One-page view

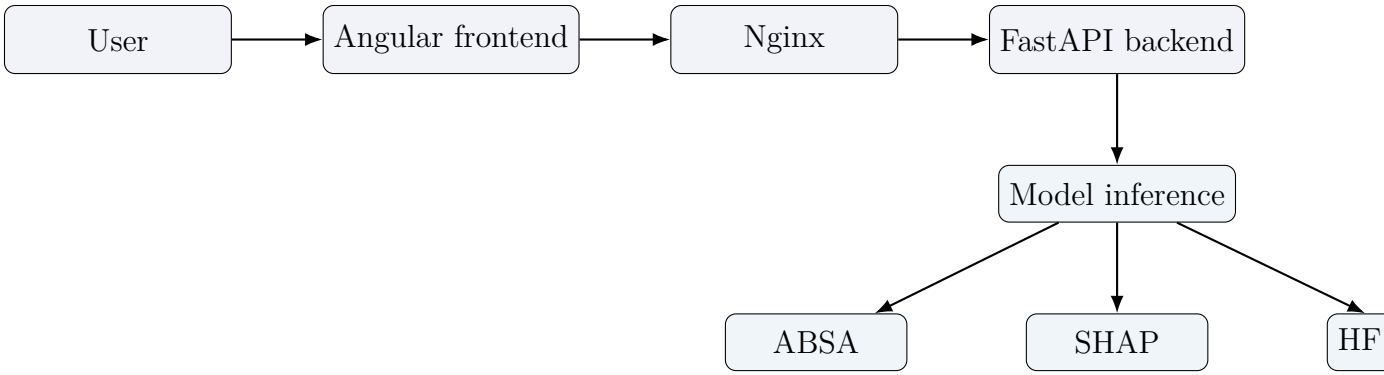

Figure 1: High-level runtime mental model

At runtime the backend is the center of gravity. It loads the model during startup, warms the ABSA pipeline, exposes health and metrics, and answers all prediction requests. The rest of the stack exists to make that service usable, observable, and reproducible.

3.2 Engineer view

- **Frontend:** message composer, history, explain panel, batch upload modal
- **Backend:** FastAPI routes, validation, error mapping, metrics middleware
- **Model layer:** baseline HF model, finetuned PEFT model, ONNX runtime
- **Offline ML:** data download, preprocessing, validation, finetuning, export
- **Operations:** Prometheus, Grafana, MLflow, Docker Compose, Nginx

4 Architecture Deep Dive

4.1 Runtime architecture

Table 2: Major components and responsibilities

Component	Primary file(s)	Responsibility
Frontend	`app/sentiment-analysis-chatbot/src/app/*`	User interaction, state, API calls, explainability UI
Backend	`src/main.py`	Request validation, routing, startup, error handling, metrics
Model	`src/model/baseline.py`	Sentiment, sarcasm, ABSA, SHAP, batch prediction
ONNX	`src/model/onnx_inference.py`	Runtime session loading and optimized inference
Training	`src/scripts/run_finetuning.py`, `src/training/*`	Adapter training, class weighting, MLflow tags
Data	`src/data/*`	Download, preprocess, validate, split, report
Monitoring	`src/monitoring/*`, `infra/*`	Metrics, dashboards, alerts
Deployment	`docker-compose.yml`, `Dockerfile`	Service orchestration and packaging

4.2 Design tradeoffs

- one backend service is easier to operate than a microservice mesh
- a single model abstraction avoids leaking training/runtime details into the API layer
- ONNX is an optimization, not the source of truth; the training code still lives in PyTorch/Hugging Face
- ABSA and SHAP are valuable, but they are not free; the code keeps them explicit so callers can choose when to pay their cost

5 Runtime Execution Flow

5.1 Application startup

- `src/main.py` creates the FastAPI app and registers a lifespan handler
- the lifespan handler reads `MODEL_MODE`
- it constructs `ModelConfig`
- it instantiates `BaselineModelInference`
- it calls `preload()` so ABSA is warm before traffic arrives

5.2 Prediction flow

1. the user types text in `ChatInputComponent`
2. the frontend service posts the text to `/predict`
3. `src/main.py` validates the request with `PredictRequest`
4. language is resolved by the backend helper
5. `BaselineModelInference.predict_single()` runs the model
6. the response is assembled into `PredictResponse`
7. the UI renders sentiment, confidence, aspects, and sarcasm

5.3 Other flows

- **Health check:** verifies the model is loaded, not the entire platform

- **Explain:** finds the previous user message and calls `/explain`
- **Batch:** parses CSV, validates the text column, predicts row by row
- **Evaluation:** background task and CLI both reuse the same model abstraction

6 Frontend Deep Dive

6.1 Component hierarchy

- `AppComponent`: app shell, health polling, chat history, modal state
- `SentimentAnalysisService`: HTTP client and message state
- `ChatInputComponent`: text input, language selection, voice input
- `MessageBubbleComponent`: renders bot/user messages and SHAP results
- `BatchUploadComponent`: uploads CSV files and exports results

6.2 State and storage

- chat history is stored in local storage
- dark mode is also stored locally
- the service maintains the active message list as the source for UI rendering

6.3 Frontend/backend contract risks

- the frontend language selector is not wired through end to end
- batch upload validation is only superficial on the client
- the UI assumes the response schema stays stable; changes in labels or fields must be coordinated

7 Backend Deep Dive

7.1 API surface

Table 3: Implemented endpoints

Method	Path	Purpose
GET	`/health`	Readiness and model state
POST	`/predict`	Single-text prediction
POST	`/explain`	SHAP explanation
POST	`/batch_predict`	CSV batch prediction
GET	`/batch_status/{job_id}`	Mocked status endpoint
POST	`/evaluate`	Background evaluation trigger
GET	`/evaluate/status`	In-memory evaluation status
GET	`/metrics`	Prometheus scrape endpoint

7.2 Error handling

- input validation is done with Pydantic schemas
- model errors are converted into JSON responses or 4xx/5xx errors
- `/batch_status` is mocked and does not back a real job queue

8 AI Inference Deep Dive

8.1 What the model layer does

The model layer hides multiple inference modes behind one interface. That is the key architectural choice. The API layer does not need to know whether the runtime is using a baseline HF checkpoint, a finetuned PEFT adapter, or an ONNX session.

8.2 Core concepts

- **Tokenizer:** converts text into tokens, ids, attention masks, and tensors
- **Logits:** raw scores from the classifier head before softmax
- **Softmax:** turns logits into probabilities that sum to 1
- **Confidence:** the probability associated with the chosen label
- **LoRA / PEFT:** lightweight adapters added to the base backbone
- **ABSA:** extracts aspect-level sentiment, such as food or service

- **SHAP:** token-level explanation of why the model chose a label
- **Sarcasm:** a separate flag that captures irony-like behavior

8.3 Production implications

- ONNX improves deployment predictability and can reduce latency
- ABSA and SHAP should remain explicit opt-in costs
- batch prediction skips ABSA because row-wise ABSA would be too expensive
- finetuned adapters must be versioned with data and code

9 ML Training Deep Dive

9.1 Training pipeline

- data is downloaded by task
- raw files are normalized and split
- validation enforces expected shape and label quality
- finetuning trains sentiment and sarcasm adapters with PEFT/LoRA
- evaluation logs metrics to MLflow
- export merges adapters and produces ONNX artifacts

9.2 Balancing and loss

- class imbalance is handled with weighted loss
- oversampling is used where it improves minority representation
- smoke mode exists for fast verification, not for quality benchmarks

10 Data Lineage

Table 4: Key artifacts

Artifact	Producer	Consumer	Lifecycle
`data/raw/*`	Downloader scripts	Preprocessing / training	Pulled from external datasets
`data/processed/*`	Preprocess pipeline	Baseline evaluation / training	Derived, versioned by DVC
`models/adapters/*`	Finetuning scripts	ONNX export / runtime	Training output
`models/onnx/*`	ONNX exporter	Runtime inference	Deployable artifact
`data/reports/*`	Validation / evaluation scripts	Humans / MLflow	Audit and review artifact

11 DVC Pipeline

- `download`: fetch raw corpora
- `preprocess`: clean, normalize, split
- `validate`: enforce quality gates
- `evaluate_baseline`: score baseline model

- `download_sarcasm` and `download_sentiment`: prepare task data
- `finetune_sarcasm` and `finetune_sentiment`: train adapters
- `evaluate_finetuned`: compare finetuned results
- `export_onnx_sentiment` and `export_onnx_sarcasm`: package runtime artifacts
- `benchmark_onnx`: measure runtime behavior

12 ONNX Pipeline

ONNX exists to reduce runtime friction. The exporter merges PEFT adapters into the base model, exports a static graph, and optionally produces an INT8 variant. The runtime loader then chooses a provider based on available hardware. That makes the serving path more predictable than shipping the whole finetuning stack to production.

13 API Reference

Table 5: API details

Method	Path	Request / response	Notes
GET	`/health`	No body / health schema	Readiness only
POST	`/predict`	`PredictRequest` / `PredictResponse`	Main user path
POST	`/explain`	`ExplainRequest` / `ExplainResponse`	Expensive on demand path
POST	`/batch_predict`	multipart file / batch schema	Synchronous batch processing
GET	`/batch_status/{job_id}`	path param / mocked status	Not a real queue
POST	`/evaluate`	No body / status JSON	Background task trigger

14 Security Review

- there is no authentication or authorization
- CORS is permissive
- batch upload accepts user-supplied CSV files
- build-time secrets are a risk if they are passed as arguments

- default Grafana credentials are unsafe for production

15 Observability

- backend exposes Prometheus metrics
- Grafana dashboards visualize request rate, error rate, and latency
- alert rules cover high error rate and high inference latency
- tracing is not implemented in the repo

16 Performance Analysis

- startup cost is dominated by model loading and ABSA warm-up
- SHAP and ABSA are the most expensive request-time features
- batch throughput is limited because the model still executes per-row inference
- memory usage scales with backbone size and adapter state

17 Failure Analysis

- **startup failure:** model path or artifact missing
- **prediction 500:** tokenizer/runtime/model mismatch
- **explain failure:** previous user message not found or SHAP error
- **batch failure:** malformed CSV, missing text column, or upload limit
- **metrics missing:** middleware or scrape configuration issue

18 Technical Debt Register

- mocked batch status endpoint
- frontend language selector not wired through
- docs and implementation drift in a few places
- no auth or rate limiting
- frontend test coverage is weak

19 Ownership Matrix

Table 6: Ownership summary

Lead	Main ownership area
Tech Lead	architecture, roadmap, security, production readiness
ML Engineering Lead	training, datasets, evaluation, DVC, MLflow
AI Engineering Lead	inference, ONNX, SHAP, ABSA, language detection
Solution Engineering Lead	end-to-end solution, monitoring, deployment
Backend Engineering Lead	FastAPI, validation, batch and evaluation APIs
UI/UX Engineering Lead	frontend UX, explainability, batch upload, visualization

20 RACI Matrix

- the tech lead is accountable for production readiness
- the ML lead is accountable for training and evaluation quality
- the AI lead is accountable for inference correctness and runtime performance
- the backend lead is accountable for API contracts and request flow
- the UI/UX lead is accountable for user flow and visual clarity
- the solution lead is accountable for end-to-end integration

21 Team Responsibilities

The project is easiest to maintain when each function has a clear owner. Cross-cutting changes, such as adding a new label, a new language, or a new runtime mode, should be reviewed by backend, AI, ML, and UI owners together because those changes touch contracts, training data, inference code, and the frontend.

22 Debugging Handbook

22.1 Backend does not start

Check `MODEL_MODE`, artifact paths, DVC-pulled files, environment variables, and startup logs.

22.2 Prediction fails

Check request validation, supported language, tokenizer availability, model mode, and ONNX session state.

22.3 Batch upload fails

Check CSV shape, the `text` column, file size, and Nginx upload limits.

23 Incident Response Guide

1. identify whether the failure is startup, request, or data related
2. inspect backend logs first because the backend owns model loading and request handling
3. verify metrics and health endpoints
4. compare the runtime artifact version against the last successful training or export run
5. roll back model artifacts before changing application code if the issue is runtime only

24 New Engineer Onboarding Guide

- read `src/main.py`, `contracts/schemas.py`, and `src/model/baseline.py` first
- then read the frontend service and the batch component
- next read the training and ONNX export scripts
- finally inspect DVC, Docker, and monitoring configs

25 Production Readiness Assessment

This repository is close to a production-style ML application, but it still has several gaps: no auth, no rate limiting, a mocked batch status endpoint, and a few contract mismatches. The architecture is directionally good, but production hardening is still needed.

26 Roadmap

Table 7: Suggested roadmap

Horizon	Focus
3 months	close contract gaps, secure the API, fix batch semantics, harden tests
6 months	improve observability, add traceability, refine ONNX and evaluation workflows
12 months	scale the runtime topology, formalize artifact provenance, and tighten ownership boundaries